Ders 7 Genişletilmiş Notu: Kelime Gösterimleri, Kelime Gömmeleri ve Metin Sınıflandırma

Ders 7 Genişletilmiş Notu: Kelime Gösterimleri, Kelime Gömmeleri ve Metin Sınıflandırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın. Altyazıda okunmayan bazı slayt içerikleri (Seattle-Denver bağlam tablosu, PMI benzerlik tablosu, hazır vektör tablosu, kelime koordinatları matrisi, AFE terim-doküman matrisi) kısa nottan alınmıştır; o not yazılırken ders karelerine bakılmıştı.

İçindekiler

- 1. Dersin Planı
- 2. Kelime Gösterimi Nedir
- 3. Vektör Uzay Modeli, Uzaklık ve Benzerlik
- 4. Count, Terim Frekansı ve Normalizasyon
- 5. Vektörlerle Analoji ve Eğiticisiz Öğrenme
- 6. One-Hot Gösterimi
- 7. Doküman, Bağlam ve Trigram Tabanlı Gösterimler
- 8. İlgililik ve Cosine Benzerliği
- 9. Seattle-Denver Örneği
- 10. Pointwise Mutual Information
- 11. Kelime Analojileri
- 12. Kelime Gömmeleri
- 13. word2vec, Skip-gram ve CBOW
- 14. FastText
- 15. GloVe
- 16. Hazır Eğitilmiş Kelime Vektörleri
- 17. BERT
- 18. Makine Öğrenmesinin NLP Açısından Hızlı Tekrarı
- 19. Özellik Seçimi ve Özellik Çıkarımı
- 20. Sınıflandırma Yöntemleri ve Çapraz Geçerleme
- 21. Kümeleme ve Yarı Eğiticili Yöntemler
- 22. Metin Sınıflandırma
- 23. Yazar Tanıma Özellikleri
- 24. Metin Gösterimi ve Doküman-Terim Matrisi
- 25. Hiyerarşik Metin Kümeleme
- 26. Boyutun Laneti ve Özellik Kararları
- 27. Kelime Torbası
- 28. Abstract Feature Extraction
- 29. Kelime Koordinatları Yöntemi
- 30. TF-IDF
- 31. İzdüşüm Yöntemleri
- 32. Naive Bayes ile Metin Sınıflandırma

1. Dersin Planı

Ders iki bölümden oluşuyor. Birinci bölümde kelime gösterimleri (word representation) ve kelime gömmeleri (word embeddings) anlatılıyor. Aradan sonra, makine öğrenmesi konularının NLP açısından önemli yerleri hızlıca hatırlatılıyor ve metin sınıflandırmaya (text categorization) geçiliyor.

Hoca, makine öğrenmesi dersini almayan varsa sohbete yazmasını istedi. Kimse yazmadığı için herkesin bu dersi aldığını kabul etti ve o slaytları ayrıntıya girmeden geçti. Yalnız NLP'de bilinmesi gereken yerlerde durdu.

2. Kelime Gösterimi Nedir

Bir metin ya da dokümanla işlem yapılacaksa, ondan birtakım özellikler çıkarılmalı ve bu özellikler vektör hâline getirilmelidir. Hangi özelliklerin alınacağı yapılan uygulamaya göre değişir. Örneğin metin sınıflandırmada başka, başka bir uygulamada başka özellikler alınır. Ama özelliklerin gösterim biçimi aynıdır. Bu bölümün konusu, neyin özellik olacağı değil, nasıl gösterileceğidir.

Bir kelime, cümledeki kullanımına göre farklı anlamlar kazanabilir. Bu yüzden kelimeler bazen anlamları da hesaba katılacak biçimde, bazen de yalnızca sayısal varlıklarıyla gösterilir. Kelimelerin işlenebilir formattaki hâline **kelime gösterimi** denir.

Kelime gösterimleri iki ana gruba ayrılır:

- **Klasik, frekans tabanlı gösterimler:** count vektörü ve TF-IDF (Term Frequency - Inverse Document Frequency).
- **Tahmin tabanlı gösterimler:** Hayatımıza derin öğrenme yaklaşımıyla girdi. En çok kullanılanlardan biri word2vec'tir. Bunun yanında GloVe, BERT, ELMo, BERT'ten türetilen modeller ve FastText vardır.

Eskiden de yapay sinir ağları vardı ve genelde bir girdi (input), bir gizli (hidden) ve bir çıktı (output) katmanından oluşuyordu. Gizli katmanların derinliği arttıkça derin öğrenme karşımıza çıktı.

Hoca burada slayttaki bir sözü okudu: bilimin değeri işleri karmaşıklaştırmak değil, özündeki basitliği bulmaktır. Kelime gösterimlerinde de durum böyledir. Arka planda çok karmaşık işler yapılabilir, sayısal değerlerin çıkarılması çok zaman alabilir. Ama sonuçta elde edilen şey basit bir vektördür. Bu vektör bir kelimeyi, bir cümleyi ya da bir dokümanı anlatır ve içindeki sayılar önemli bilgiler verir.

3. Vektör Uzay Modeli, Uzaklık ve Benzerlik

Vektör uzay modelinde (vector space model) elimizde bir kelimeyi, cümleyi, paragrafı ya da dokümanı ifade eden vektörler vardır. Her vektör uzayda bir noktaya karşılık gelir. Vektör 100 elemanlıysa, 100 boyutlu uzayda bir noktadır. Bu model bilgi erişimi (information retrieval) konusunda da karşımıza çıkacak.

Amaç iki vektör arasındaki benzerliği bulmaktır. Hoca bunu iki boyutta çizerek gösterdi. İki vektörün benzerliğini bulmak, aralarındaki açıyı bulmak demektir. Açı hesabı maliyetli olduğu için bu işi kolaylaştıran metrikler geliştirilmiştir. Böylece karşımıza iki kavram çıkar: **uzaklık** (distance) ve **benzerlik** (similarity). Bu ikisi birbiriyle ters orantılıdır.

İki değer $[0, 1]$ kapalı aralığına alındığında:

- Uzaklık 0 ise benzerlik 1'dir. Aralarında hiç mesafe yok, iki vektör birbirinin aynısıdır.
- Benzerlik 0 ise uzaklık 1'dir.

Vektörler birbirinden uzaklaştıkça uzaklık artar, benzerlik azalır.

4. Count, Terim Frekansı ve Normalizasyon

Vektörün içindeki sayılar farklı anlamlar taşıyabilir. Hoca bunu bir cümlenin karakter vektörüyle anlattı.

Önce şu soru sorulur: büyük-küçük harf ayrımı benim için önemli mi? Önemli değilse her şey küçük harfe çevrilir. Türkçede 29 harf olduğu için 29 boyutlu bir vektör alınır.

- **Count:** Her harfin cümlede kaç kez geçtiğidir. "a" 5 kez geçtiyse "a" gözüne 5 yazılır.
- **TF (terim frekansı):** Count değerini doğrudan kullanmaz. En sık geçen terimin sayısına böler. En sık geçen "e" 10 kez geçtiyse, 5 kez geçen terim için $5/10$, "e" için $10/10$ yazılır. Böylece değerler 0 ile en sık terimin değeri arasına sıkışır, çok dağılmış olarak durmaz.

Normalizasyonun neden önemli olduğunu hoca şöyle gösterdi. Boşluk da 30. özellik olsun. Bir dokümanda 10.250 boşluk olabilir, diğer karakterler ise 3, 5, 7 civarında olabilir. Arada çok büyük bir fark vardır. Benzerlik ve uzaklık hesaplanırken toplama, çarpma ve kare alma yapılacaktır. Bir yerde 5'in karesi, başka bir yerde 10 binin karesi alınırsa yanıltıcı sonuçlar çıkar. Vektörler normalize edilerek oluşturulursa sonuçlar belirli bir aralıkta kalır ve daha etkili olur.

> **Hocanın vurgusu:** Vektörlerinizi hep normalize ederek oluşturursanız daha etkili sonuçlar alırsınız.

5. Vektörlerle Analoji ve Eğiticisiz Öğrenme

Vektörler varsa kelimeler arasındaki ilişkiler de bulunabilir. Elimizde erkek ve kadın için çıkarılmış vektörler, bir de kral vektörü olsun. Kraliçe vektörü yoksa ne olabilir? Erkek ile kadın vektörleri arasındaki fark, kral ile kraliçe arasındaki farka eşit kabul edilir. Sanki cebirsel işlem yapılıyormuş gibi düşünülür.

Kelime vektörleri arasındaki mesafe için çok sayıda uzaklık ve benzerlik metriği vardır. Ama metinlerde cosine benzerliği oldukça başarılı olduğu için genelde bu kullanılır. Akademik çalışmalarda da hep cosine benzerliği görülür.

word2vec, GloVe, BERT gibi derin öğrenme modelleriyle kelime vektörlerinin öğrenilmesi **eğiticisiz** (unsupervised) bir iştir. Sınıf bilgisine hiç gerek yoktur. Her kelime için belirli sayıda özellik öğrenilir. Bu sayıya kullanıcı karar verir. Her özellik anlamsal bir bilgi taşır.

6. One-Hot Gösterimi

En basit gösterim one-hot gösterimidir. Derin öğrenme modellerinin girdi katmanına da vektörün one-hot hâli verilir.

Bir sözlüğümüz olsun. Sözlük (vocabulary) boyutu $V = 10.000$ ise her kelime 10.000 boyutlu bir vektördür:

- "a" sözlükte birinci sıradaysa vektörün birinci gözü 1, diğerleri 0 olur.
- "adam" ikinci sıradaysa ikinci gözü 1, diğerleri 0 olur.
- "zil" 10.000. kelimeyse son gözü 1, diğerleri 0 olur.

Görüldüğü gibi bu gösterim seyrek (sparse) bir veridir.

7. Doküman, Bağlam ve Trigram Tabanlı Gösterimler

Hoca, çok değişik gösterimler türetilebileceğini, hatta uygulamaya göre kendimizin de farklı bir gösterim kullanabileceğimizi söyledi. Önemli olan, kelimeyi anlatabilecek bir gösterim olmasıdır.

> **Hocanın vurgusu:** "Hangisini seçeceğiz?" sorusunun cevabı şudur: her biriyle çalışılır, en iyi performansı hangi gösterim verirse o seçilir. Hepsi uygulamaya bağlıdır.

7.1 Doküman tabanlı gösterim

Kelime, geçtiği dokümanlara göre gösterilir. 12 doküman olsun ve "aba" kelimesinin vektörü oluşturulsun. Kelime 1. dokümanda hiç, 2. dokümanda 1, 4. dokümanda 1, 7. dokümanda 2, 9. dokümanda 1, 11. dokümanda 1 kez geçmiş olsun. Bu sayılar vektörün gözlerine yazılır. Gözlerde count ya da TF değerleri kullanılabilir.

7.2 Bağlam (komşu kelime) tabanlı gösterim

Cümle “Yellow banana grows on trees in Africa” olsun ve “banana” için vektör oluşturulsun. Kelimenin solundaki 5 ve sağındaki 5 kelimeye bakılır. Her göz bir (kelime, konum) çiftidir:

- yellow: 1 solda (-1)
- grows: 1 sağda (+1)
- on: 2 sağda (+2)
- trees: 3 sağda (+3)
- in: 4 sağda (+4)
- Africa: 5 sağda (+5)

Dokümanın başka bir yerinde de “banana grows” geçiyorsa (grows, +1) gözünün değeri 2 olur. Bu bize, banana kelimesinin cümleler içinde en çok “grows” ile birlikte geçtiğini söyler. Başka bir cümlede yine “yellow banana” olsaydı (yellow, -1) gözü de 2 olurdu.

Vektör aslında çok daha uzundur. Başka bir kelime, örneğin “an orange” içindeki “orange” için (an, -1) gözü 1 olur, banana için aynı göz 0’dır. Yani kelime, hangi kelimelerle daha çok kullanılıyorsa ona göre bir sayım oluşturulur.

7.3 Trigram tabanlı gösterim

Vektörün gözleri, Türkçede en sık kullanılan ilk 100 ya da ilk 500 trigram olabilir. “banana” kelimesinin trigramları sırasıyla “ban”, “ana”, “nan”, “ana”, “na_” olur (_ boşluk). “ana” iki kez geçtiği için gözüne 2 yazılır. Aynı şey bigramlarla ya da başka birimlerle de yapılabilir.

8. İlgililik ve Cosine Benzerliği

İki kelimenin vektörleri karşılaştırılarak ne kadar benzer oldukları bulunur. İlgililik (relatedness) kavramı, seçilen gösterimle çok yakından ilişkilidir.

One-hot gösterimiyle kelimeler arasındaki benzerlik bulunamaz. Hiçbir kelime hiçbir kelimeye benzemez. Bağlam vektörleriyle ise benzerlik çıkarılabilir, çünkü aynı kelimelerle kullanılan kelimeler birbirine yakın olabilir. Afrika’da ağaçta yetişen sarı bir meyve banana ise, Afrika’da yetişen başka bir meyve için de yine “yetişmek”, “ağaç”, “Afrika” kelimeleri geçer. Demek ki bunlar aynı konseptte geçen kelimelerdir.

Metinlerde en önemli metrik cosine benzerliğidir:

$$\cos\theta = \frac{\vec{a}\cdot\vec{b}}{\|\vec{a}\|\|\vec{b}\|}$$

Pay, iki vektörün skaler çarpımıdır, yani karşılıklı elemanların çarpımlarının toplamıdır. Payda, vektör boylarının çarpımıdır. Bir vektörün boyu, elemanlarının karelerinin toplamının karekökü dür.

Kral-kraliçe vektörleri arasındaki benzerlik ile erkek-kadın vektörleri arasındaki benzerlik birbirine yakın çıkar. Buradan kralın erkek, kraliçenin kadın olduğu çıkarılabilir. Erkek ile kadın arasında bir zıtlık ilişkisi varsa, kral ile kraliçe arasında da aynı ilişki vardır.

9. Seattle-Denver Örneği

Dört doküman olsun. Her cümle bir doküman kabul edilir; doküman ya da cümle olması fark etmez, hepsi birer metin parçasıdır.

- Doküman 1: “seattle seahawks jerseys”
- Doküman 2: “seattle seahawks highlights”
- Doküman 3: “denver broncos jerseys”
- Doküman 4: “denver broncos highlights”

Soru şudur: Seattle ile Denver, Seahawks ile Broncos birbirine benzer kelimeler midir? Aralarında anlamsal bir benzerlik var mı?

9.1 Konu benzerliği (topic similarity)

Kelimenin dokümanda geçip geçmediği bilgisi kullanılır. Dört doküman olduğu için vektörler 4 boyutludur.

Kelime	D1	D2	D3	D4
seattle	1	1	0	0
seahawks	1	1	0	0
denver	0	0	1	1
broncos	0	0	1	1

seattle ile seahawks arasındaki cosine benzerliği adım adım şöyledir:

$$\frac{1 \cdot 1 + 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0}{\sqrt{2}\sqrt{2}} = \frac{2}{2} = 1$$

Benzerliğin 1 çıkması, bu gösterimde iki vektörün birbirinin aynısı olduğu anlamına gelir. denver ile broncos için de aynı sonuç çıkar. Yani bu gösterim, birlikte geçen kelimeleri örtüşen kelimeler olarak bulur.

9.2 Bağlam vektörleri

Bu kez kelimenin 2 solu ve 2 sağı kullanılır.

Kelime	(seattle,-1)	(seahawks,-1)	(denver,-1)	(broncos,-1)	(jerseys,+1)	(jerseys,+2)	(highlights,+1)	(highlights,+2)
seattle	0	2	0	0	0	1	0	1
seahawks	2	0	0	0	1	0	1	0
denver	0	0	0	2	0	1	0	1
broncos	0	0	2	0	1	0	1	0

Hoca değerlerin birbirinin aynısı olmadığını, ama benzerlik ölçüldüğünde seahawks'ın broncos'a, seattle'ın denver'a daha yakın çıktığını söyledi. Tablodan hesaplanınca (ara adım eklendi): seattle ile denver için skaler çarpım $1 + 1 = 2$, boylar $\sqrt{6}$, benzerlik $2/6 = 1/3$ olur. seahawks ile broncos için de $1/3$ çıkar. seattle ile seahawks arasında ise ortak göz olmadığı için benzerlik 0'dır.

Sonuç önemlidir. Birinci gösterimde seattle ile seahawks benzer çıkarken, ikinci gösterimde seattle ile denver benzer çıkıyor. İkisi de şehir olduğu için bu da ayrı bir benzerliktir.

> **Hocanın vurgusu:** Vektörleri nasıl oluşturduğunuz bulduğunuz benzerliği de değiştirir. Hangisini seçeceğiniz uygulamanızın amacına bağlıdır.

Aynı işlem trigramlarla da yapılabilir. "seattle" ile "settle" kelimelerinin trigramları vektörün gözlerine yerleştirilir ve iki vektörün cosine benzerliği ölçülür.

10. Pointwise Mutual Information

Cosine benzerliğinden başka metrikler de vardır. Sık kullanılanlardan biri Pointwise Mutual Information'dır (PMI). Olasılık kullanarak iki kelimenin benzerliğini ölçer:

$$pmi(x; y) = \log \frac{p(x, y)}{p(x)p(y)}$$

İki kelimenin birlikte geçme olasılığı, ayrı ayrı olasılıklarının çarpımına bölünür ve logaritması alınır.

Hoca internette de bulunabilecek bir uygulamayı gösterdi. Bir kelime giriliyor, PMI ile benzerlik katsayısı hesaplanıyor ve en yakın kelimeler listeleniyor. Kelimenin kendisiyle benzerliği 1 olarak listenin başında görünüyor.

Sorgu	En benzer kelimeler (benzerlik katsayısı)
sydney	melbourne 0,44, brisbane 0,41, perth 0,34, adelaide 0,29, auckland 0,25
batman	spiderman 0,14, superman 0,14, ghostbusters 0,10, tinkerbell 0,09, starwars 0,08
java	c 0,16, javascript 0,15, powershell 0,11, python 0,10, vb 0,09
pasta	spaghetti 0,18, lasagna 0,15, macaroni 0,11, salad 0,10, casserole 0,10

Değerler 1'e yakın değildir. Ama çıkan kelimeler hep aynı alandandır: Avustralya şehirleri, çizgi roman ve film kahramanları, programlama dilleri, yemekler. "Sydney" için "book" ya da "pencil", "pasta" için "table" gibi kelimeler çıkmıyor. Benzerlikler birlikte geçme olasılıklarından çıktığı için kullanılan derleme göre değişir.

11. Kelime Analojileri

Kelimeler arasındaki benzerliklere bakarak analoji de çıkarılabilir.

- King ile Queen arasında bir ilişki varsa, Man ile Woman arasında da aynı ilişki vardır.
- Bu yalnızca isimlerde değil fiillerde de geçerlidir: walking-walked ile swimming-swam.
- Ülke-başkent ilişkisi: Spain-Madrid verilince Italy için Rome, China için Beijing bulunur.

Kullanılan işlem şudur:

$$\vec{king} - \vec{man} + \vec{woman} \approx \vec{queen}$$

Bu işlem tam olarak queen vektörünü vermez. Queen'in kendi vektörüne çok yaklaşan bir vektör verir. Bu yüzden "yaklaşık" ifadesi kullanılır.

12. Kelime Gömmeleri

Kelime gömmeleri (word embeddings) son yıllarda hayatımıza çok girdi. Hoca Türkçe "gömme" kelimesini komik bulduğunu, hep "embedding" dediğini söyledi.

Şimdiye kadar görülen vektörlerin boyutları çok yüksekti, sözlüğün boyutu kadardı ve çok seyrekti. Kelime-doküman matrisleri de seyrekti. Aynı sezgileri kullanan ama **daha düşük boyutlu ve içerik olarak daha yoğun** vektörler geliştirilmeye başlandı.

- **Düşük boyutlu:** Boyuta kullanıcı karar verir: 100, 150, 300 gibi. Boyut çok küçük tutulursa başarı düşer. Çok yüksek tutulursa anlamsal ilişkiler kaybolabilir. Çalışmalarda genelde 150 ve 300 tercih edilir.
- **Yoğun (dense):** Her gözde sıfırdan farklı bir değer vardır ve vektörün elemanları arasındaki ilişkiler çok kuvvetlidir.

Word embedding kavramı 2013'te Google'da çalışan Mikolov ve ekibi tarafından ortaya atıldı. Hocaya göre arka planda bir matris ayrıştırma (matrix factorization) yapılır, yani matris anlamlı parçalara bölünmeye çalışılır. Arkasında bir yapay sinir ağı vardır: girdi katmanı, gizli katman(lar) ve çıktı katmanı. Bu yapı hem encoder hem de decoder gibi çalışır.

Hoca modellerin içine girmedi. Bunun benzerleri derin öğrenme dersinde anlatılıyor. Pratikte Python'da model hazırdır, katmanlar eklenip çalıştırılır.

12.1 Bilgi erişiminde kullanım

Embedding'ler çok yerde kullanılır. Hoca ilginç bulduğu bir örnek verdi. Geleneksel bilgi erişim sistemleri terim eşleştirme (term matching) yapar. Sorgu kelimesi bir pasajda geçiyorsa o pasajı kullanıcıya getirir. Oysa pasajlardan biri gerçekten o konudayken, diğerinde kelime yalnızca bir ad olarak geçiyor olabilir. Embedding kullanılırsa dokümanların içerikçe yakınlığı da kontrol edildiği için ikinci pasaj getirilmez.

12.2 Embedding matrisi

Sözlükte M kelime (örneğin 10.000) ve her kelime için D boyutlu vektör varsa, her kelimenin vektörü bir sütuna yerleştirilir. Böylece D satır, M sütunluk bir matris elde edilir. Bir cümle de bu vektörlerle matris hâline getirilebilir. Vektörü olmayan kelimelerin yerine belki sıfır konur.

En bilinen embedding yöntemleri word2vec, GloVe ve BERT'tir. Bunlardan gelişmiş modeller de vardır. Ayrıntılar çıkış makalelerinden okunabilir.

13. word2vec, Skip-gram ve CBOW

word2vec kelimeyi vektörle ifade eder. İki modeli vardır: CBOW (Continuous Bag of Words) ve skip-gram. İkisi aynı yapıdadır ama ters yönde çalışır.

13.1 Temel kavramlar

- **Merkez kelime:** Embedding'i oluşturulacak kelime. Slayttaki örnekte "şafaklarda".
- **Çerçeve (pencere, window) boyutu:** Merkez kelimenin sağından ve solundan kaç kelime alınacağını belirler. Soldan 3, sağdan 3 kelime alınırsa pencere boyutu 7 olur.
- **Çevreleyen kelimeler:** Merkez kelimenin etrafındaki kelimeler.

Türkçe için önemli bir soru vardır: embedding'ler kelimelerin kökünden mi, ekli hâlinden mi çıkarılmalı? "Korkma" kelimesinin kökü "kork"tur. Hocanın cevabı şudur: semantik ilişkiler de yakalanmak isteniyorsa ne yazık ki ekleriyle birlikte çıkarılmalıdır. Bu yüzden Türkçede embedding sayısı oldukça fazladır.

13.2 Ağın yapısı

word2vec bir girdi, bir gizli ve bir çıktı katmanından oluşan bir yapay sinir ağıdır. Hiperparametreleri pencere genişliği ve embedding boyutu gibi değerlerdir.

Bütün kelimelerin vektör boyutu aynı olmak zorundadır. Embedding boyutu, gizli katmandaki nöron sayısına eşittir. Slayttaki örnekte:

- Girdi one-hot vektördür. Sözlük boyutu 10.000'dir ve "ants" kelimesinin gözü 1'dir.
- Gizli katmanda 300 nöron vardır.
- Çıktıda softmax sınıflandırıcı ve 10.000 nöron vardır. Her nöron sözlükteki bir kelimeye karşılık gelir.

Kullanım alanları arasında iki kelimenin ya da iki cümlenin benzerliğini bulmak ve doküman özetleme vardır.

13.3 Skip-gram

Skip-gram'da girdi merkez kelime, çıktılar çevresindeki kelimelerdir.

Örnek cümle: "Korkma sönmez bu şafaklarda yüzen al sancak". "Şafaklarda" kelimesinin embedding'i bulunmak isteniyor. Hoca çizerken pencereyi 5 aldı: ortada "şafaklarda", solunda "sönmez" ve "bu", sağında "yüzen" ve "al". (Slayttaki tanımda ise pencere 7'dir; o durumda "korkma" ve "sancak" da eklenir.)

Girdide "şafaklarda" kelimesinin one-hot vektörü, çıktılarda çevreleyen kelimelerin one-hot vektörleri bulunur. Ortada boş bir vektör vardır ve boyutu ne seçildiyse odur. Ağ çalışır ve "şafaklarda" için bir vektör oluşturur.

Metinde ilerlenir. “Şafaklarda” başka bir yerde yine geçerse, oradaki sağ ve sol kelimelere göre vektör yeniden güncellenir. Bütün derlem bitince, bu kelimenin derlemdeki kullanımına bağlı sayısal değerlerden oluşan bir vektör çıkar. Sıra “yüzen” kelimesine geldiğinde girdi onun one-hot hâli, çıktılar da “bu”, “şafaklarda”, “al”, “sancak” olur. Derlem bitince her kelime için bir embedding elde edilmiş olur.

Sistem aslında şunu sorar: böyle bir girdinin bu çıktıları vermesi için ortadaki vektör ne olmalıdır? Aynı girdiye başka çıktılar gelince vektör nasıl değişmelidir?

13.4 CBOW

CBOW bunun tam tersidir. Girdiler merkez kelimenin solundaki ve sağındaki kelimelerin one-hot vektörleri, çıktı ise merkez kelimenin one-hot vektörüdür. Seçilen boyuttaki vektör yine ortada oluşur.

13.5 Karşılaştırma

Hocanın verdiği karşılaştırma şöyledir:

Özellik	CBOW	Skip-gram
Veri seti	Küçük veri setlerinde daha iyi	Büyük veri setlerinde daha iyi
İşlem gücü	Az	Daha fazla
Çok anlamlı kelimeler (“yüz”)	İyi embedding oluşturamıyor	Daha iyi embedding

> **Ders sonrası not:** Veri seti satırı, word2vec’in özgün açıklamasındaki ifadenin tersidir. Orada skip-gram’ın az veriyle iyi çalıştığı ve nadir kelimeleri iyi temsil ettiği, CBOW’un ise çok daha hızlı eğitildiği ve sık kelimelerde biraz daha iyi olduğu yazar.

14. FastText

FastText, 2016’da Facebook tarafından word2vec’in bir uzantısı olarak geliştirildi. Yapay sinir ağına girdi olarak kelimeler değil, karakter n-gram’ları verilir.

Örneğin trigram kullanılsın. “dogal” için trigramlar “dog”, “oga”, “gal” olur. Bu trigramların vektörleri vardır. Kelimenin vektörü, trigram vektörleri toplanıp ortalaması alınarak ya da arka arkaya eklenerek elde edilir.

Neden tercih edilir?

- **Yazım hatası:** “doğal” yerine yumuşak g kullanmadan “dogal” yazılsın. Böyle bir Türkçe kelime olmadığı için diğer modellerde vektörü yoktur. Ya bütün vektöre sıfır, ya da belki rastgele bir değer atanır. FastText’te ise “dog”, “oga”, “gal” trigramlarının vektörleri başka kelimelerden gelmiştir. Gerçek “doğal” kelimesinin vektörü çıkmaz, ama ona çok uzak olmayan bir vektör elde edilir. Yani metinde düzeltemediğiniz yazım hataları varsa FastText bunu bir şekilde giderebilir.
- **Derlemde hiç geçmeyen kelime:** Derlemde olmayan ama işlenen dokümanda geçen bir kelime, trigramlarına ayrılır ve onların vektörleriyle kelimeye bir vektör atanır.
- **Az geçen kelimeler:** word2vec’e göre daha iyi ifade edilir. Dokümanlarda az kelime varsa n-gram’larla embedding çıkarmak daha başarılı olabilir.

Dezavantajı süredir. N-gram sayısı kelime sayısından çok daha fazla olabilir, çünkü çok sayıda kombinasyon vardır. Eğitim uzun sürebilir.

> **Hocanın vurgusu:** Bu modelleri uygulamanıza göre seçmelisiniz.

15. GloVe

Pennington ve arkadaşlarının önerdiği GloVe’un çıkış noktası şudur:

- Skip-gram ve CBOW, kelimenin sağındaki ve solundaki kelimelerle embedding oluşturduğu için anlamsal bilgiyi yakalar. Ama iki kelimenin birlikte geçme olasılıkları gibi istatistikleri kullanmaz.
- Matris ayrıştırma yöntemleri ve doküman-kelime matrisleri istatistik bilgisini kullanır. Ama anlamsal ilişkileri yakalayamaz.

Yani bir yöntemde biri var, ötekinde öteki. GloVe, olasılık istatistiklerinden yararlanarak yeni bir amaç (objective) fonksiyonu oluşturur:

$$J(\theta) = \frac{1}{2} \sum_{i,j=1}^{W} f(P_{ij})(u_i^T v_j - \log P_{ij})^2$$

P_{ij} iki kelimenin birlikte kullanılma olasılığıdır. u_i ve v_j vektörlerdir. Transpoz, iki vektörün rahatça çarpılabilmesi için alınır. Hoca vektörlerin tam olarak neyi temsil ettiğini o an hatırlayamadığını, ama bunun kritik olmadığını söyledi, çünkü olasılıkları hesaplayan modelin kendisidir. Birlikte bulunma olasılığı yüksek olan kelimeler öğrenmede diğerlerinden daha önemli rol oynar.

> **Ek bilgi (derste anlatılmadı):** Formüldeki f bir ağırlıklandırma fonksiyonudur. Çok sık birlikte geçen kelime çiftlerinin toplam hatayı tek başına belirlemesini engeller, hiç birlikte geçmeyen çiftlerin ($P_{ij} = 0$) ise katkısını sıfırlar.

Slayttaki örnekte London ile England ilişkisi verilip Iran için karşılık sorulmuş: Londra İngiltere'nin başkenti ise İran'ın başkenti neresidir? Çıkan sonuçlar Tahran, Bağdat, Irak, Kahire, Erbil'dir. En yüksek değer Tahran'dır. Bağdat'ın bu kadar yakın çıkması, İran, Irak, Bağdat gibi kelimelerin derlemde çok birlikte geçmesinden kaynaklanır. Yine de ilk sırada doğru değer gelir.

16. Hazır Eğitilmiş Kelime Vektörleri

Kelime vektörlerini öğrenmek maliyetlidir. Çok GPU ve çok büyük derlem gerekir. Bu yüzden Türkçe dahil birçok dil için hazır vektörler paylaşılmıştır.

Kaynak	Token sayısı	Tekil kelime sayısı	Vektör boyutu
Wikipedia 2014 + Gigaword 5	6 milyar	400 bin	50, 100, 200, 300
Common Crawl 1	42 milyar	1,9 milyon	300
Common Crawl 2	840 milyar	2,2 milyon	300
Twitter (2 milyar tweet)	27 milyar	1,2 milyon	25, 50, 100, 200
Google News	100 milyar	3 milyon	300
Facebook (294 dil)			300

Common Crawl verileri, dünya genelindeki web sayfalarının taranmasıyla elde edilmiştir. Türkçe için de özel eğitilmiş kelime vektörleri vardır. Tamamen Türkçe üzerine çalışan Loodos adlı firma, BERT ve türevleri için Türkçe modelleri açık paylaşıyor; Zemberek'in bazı modüllerini de Python'a geçirip açık kaynak olarak paylaşıyor.

17. BERT

BERT, 2018'de Google tarafından yayımlandı ve transformer'larla birlikte çok popüler oldu. Açılımı Bidirectional Encoder Representations from Transformers'tır.

Hocaya göre BERT'in ürettiği vektörler GloVe ve word2vec'ten daha iyidir ve daha fazla anlamsal ilişki çıkarır. Çift yönlü olduğu için cümleyi hem soldan sağa hem sağdan sola değerlendirir. RNN mantığında cümlenin sonuna gelindiğinde baştaki kelimeyle ilişki unutulabilir. BERT'te cümlenin anlamı ve kelimeler arası ilişkiler daha iyi çıkarılır.

Eğitimde bir kitap derlemi ile 2,5 milyar kelimelik Wikipedia verisi birleştirilmiştir. BERT-Large ve BERT-Base adlı iki model vardır. Biri 16 TPU, diğeri 4 TPU üzerinde 4 gün eğitilmiştir.

Doğal dil işlemede chatbot ve metin sınıflandırma gibi problemlerde kullanılır. Genelde önceden eğitilmiş (pretrained) model alınır, probleme göre üstüne ek katmanlar eklenir ve ince ayar (fine-tuning) yapılır.

BERT'i diğerlerinden ayıran iki strateji vardır.

17.1 Masked Language Modeling (MLM)

Cümledeki kelimelerin %15'i seçilir. Bu %15'lik kısmın:

- %80'inin yerine `[MASK]` etiketi konur, yani kelime kapatılır.
- %10'unun yerine rastgele başka bir kelime konur.
- %10'u değiştirilmeden bırakılır.

Amaç, maskelenen yere hangi kelimelerin gelebileceğini tahmin etmeyi öğrenmektir.

17.2 Next Sentence Prediction (NSP)

MLM cümle içindeki kelimeler arasındaki ilişkiye odaklanır. NSP ise cümleler arasındaki ilişkiyi kullanır. Eğitimde ikili cümleler verilir ve ikinci cümlenin birincinin devamı olup olmadığı tahmin edilir.

BERT bu yüzden soru cevaplama (question answering) sistemlerinde başarılıdır. Çünkü bir sorudan sonra ne tür bir cevabın geleceği, bir cümleden sonra hangi cümlenin geleceği çıkarılabilir.

Hoca, Google'ın BERT-Base modelini herkese açmışken bazı yazılarda bunun kapandığını ve ücretli verildiğini okuduğunu da söyledi.

> **Ders sonrası not:** Google'ın yayımladığı BERT modelleri açık lisansla dağıtılmaya devam etmektedir. Ücretli olan, modeli eğitmek ya da çalıştırmak için kullanılan bulut hizmetleridir (TPU vb.).

18. Makine Öğrenmesinin NLP Açısından Hızlı Tekrarı

Tezlerde de karşımıza çıkacak iki ana yaklaşım vardır. **Eğiticili** (supervised) yöntemlerde etiket bilgisi mutlaka vardır. **Eğiticisiz** (unsupervised) yöntemlerde etiket bilgisi yoktur.

Dokümanlardan özellik vektörü çıkarılır. Özellik adayları şunlardır:

- Kelimeler ve frekansları, anlamsal ilişkiler.
- Dil modeliyle ilgili bilgiler, n-gram'lar.
- POS etiketleri: isim, fiil, sıfat.
- Gramer kuralları: kurallı cümle, devrik cümle, soru cümlesi.

Uygulama örnekleri arasında mailin spam olup olmadığını bulmak vardır. Hoca adaletle ilgili bir örnek de verdi. Çin'in bir bölgesinde pilot bir çalışma yapılmış. Son 20-30 yılda, cinayet gibi değil, hırsızlık gibi hafif suçlarda hangi delillerle hangi cezaların verildiği sisteme öğretilmiş. Aynı şartlarla bir dava geldiğinde sistem karar veriyor. Bu yalnızca bir pilot uygulamadır ve aynı cevapların verilip verilemediği denenmiştir.

18.1 Özellik sayısı ve örnek sayısı

Hocanın "çok önemli" dediği slaytta x ekseni özellik sayısıdır (10, 100, 1000, 10^4, 10^5). y ekseni ise modeli eğitmek için gereken örnek sayısıdır. Sistem teşhisi, OCR, el yazısı tanıma, metin kategorizasyonu, pazar analizi, biyoinformatik ve makine görmesi gibi alanlar grafikte farklı aralıklarda yer alır.

Bu değerler teoriktir. Pratikte bu kadar eğitim örneği bulunamayabilir. Biyoinformatikte girdi bir DNA dizilimi olduğu için özellik sayısı çok fazla olabilir, ama daha az veriyle eğitilebilir.

> **Hocanın vurgusu:** Hangi alanda çalışırsanız çalışın, özellik sayısı ile örnek sayısı arasındaki oranı yakalamanız çok önemlidir. Hepsi için aynı şartlar geçerli değildir.

19. Özellik Seçimi ve Özellik Çıkarımı

Özellik sayısı çok fazla olursa gereksiz özellikler de girer. Bu durumda iki seçenek vardır:

- **Özellik seçimi (feature selection):** En ayırt edici özellikleri seçmek.
- **Özellik çıkarımı (feature extraction):** Çalışılan uzayı değiştirmek. Veriler o özelliklerle rahatça sınıflandırılamıyorsa uzay değiştirilir. Verinin taşıdığı bilgi değişmez, sayısal değerleri değişir. Böylece veriler doğrusal bir eğriyle ya da bir kernel ile ayrılabilir.

19.1 Filtre yöntemleri

Hoca filtre (filter) yöntemlerini şöyle anlattı: 10 özellik varsa önce birinciyle sistem çalıştırılır ve başarıya bakılır, sonra ikinciyle, üçüncüyle. En başarılılar sıralanır ve seçilir. En çok kullanılanı, Weka'da da bulunan bilgi kazancıdır (information gain). Tamamen entropiye dayanır.

> **Ders sonrası not:** Filtre yöntemleri tanım gereği sınıflandırıcıyı çalıştırmaz. Özelliği, bilgi kazancı gibi istatistiksel bir ölçütle doğrudan veriden puanlar. Her özellikle sistemi ayrı ayrı çalıştırmak, tek özellikli bir sarmalayıcı yaklaşıma daha yakındır.

> **Ders sonrası not:** Hoca "entropi ne kadar büyükse o kadar iyi bir özelliktir" dedi. Entropi $H = -\sum p \log p$ ile hesaplanır. Bilgi kazancı ise özelliğin bilinmesiyle sınıf entropisinde sağlanan azalmadır. Doğrusu: bilgi kazancı ne kadar büyükse, yani özellik bilindiğinde kalan entropi ne kadar küçükse, özellik o kadar iyidir.

19.2 Sarmalayıcı yöntemler

Sarmalayıcı (wrapper) yöntemler özellik alt kümeleri oluşturur. 5 özellik varsa 1'li, 2'li, 3'lü, 4'lü ve 5'li bütün kombinasyonlar denenir. En yüksek başarıyı veren alt küme seçilir. Bütün özelliklerden başlayıp azaltarak ya da tek özelliklerden başlayıp en başarılıları birleştirerek de ilerlenebilir.

19.3 Yeni özellik çıkarımı

İki özelliği birleştirerek yeni bir özellik yapılabilir. PCA (Principal Component Analysis) ve LDA (Linear Discriminant Analysis) bu gruptadır. Hocaya göre PCA metinlerde pek iyi sonuç vermez. LDA iyi sonuç verir ama etiketli bir yöntemdir.

20. Sınıflandırma Yöntemleri ve Çapraz Geçerleme

Hangi sınıflandırma yönteminin seçileceğine baştan karar verilemez. Olasılık tabanlı, ağaç tabanlı ve kernel tabanlı yöntemlerden birer model kurulur. Veri hangi tip modele daha uygunsa onunla devam edilir.

20.1 k en yakın komşu

Yeni gelen örneğin bütün verilere uzaklığı ölçülür ve uzaklıklar kısadan uzuna sıralanır. k kaç seçildiyse o kadar komşuya bakılır. $k = 1$ ve en yakın komşu A sınıfındaysa örnek A olur. $k = 3$ ve komşuların ikisi B, biri A ise örnek B sınıfına atanır. k değerinin seçimi önemli bir parametredir.

20.2 Learning Vector Quantization

LVQ'da veriler etiketlidir. Hoca yöntemi şöyle anlattı: bir x örneğine en yakın vektör bulunur. İkisi aynı sınıf olarak etiketlenmişse vektör ödüllendirilir ve x'e yaklaştırılır. Farklıysa cezalandırılır ve x'ten uzaklaştırılır.

20.3 Çapraz geçerleme

İlk çalışmalar veriyi %70-%30 ya da %80-%20 bölerdi. Büyük parça ile eğitir, küçük parça ile test ederlerdi. Sorun şudur: eğitim kısmına çok kolay örnekler düşerse sistem ezberler ve

testte başarısız olur. Bütün zor örnekler teste düşerse, iyi modellenmiş bir sistem bile kötü görünür.

Bunu ortadan kaldırmak için k-katlı çapraz geçerleme (k-fold cross-validation) kullanılır. Veri k parçaya bölünür. Bir parça test için ayrılır, kalanlarla eğitilir ve bir başarı elde edilir. Sonra ikinci parça test için ayrılır. Bu işlem bütün parçalar için tekrarlanır. Başarıların ortalaması sistemin başarısıdır.

Farklı özellik sayısına, sınıf sayısına ve örnek sayısına sahip veri setlerinde her veri seti farklı bir yöntemde daha başarılı olabilir. Bu sayılarla başarı arasında kesin bir ilişki kurulamaz, çünkü verinin içeriği de çok önemlidir.

> **Hocanın vurgusu:** Uzaklık tabanlı, ağaç tabanlı, kernel tabanlı ve istatistik tabanlı yöntemleri deneyip en iyisini bulmak gerekir.

21. Kümeleme ve Yarı Eğiticili Yöntemler

21.1 k-means

Küme sayısı k kullanıcı tarafından belirlenir. Etiketler bilinmese de tahmini bir sayı verilebilir. Cinsiyete göre ayırmak için 2 ya da 3 küme yeter. Etiketsiz bir doküman topluluğu için gazeteyi düşünebiliriz: ekonomi, siyaset, teknik, magazin, sağlık gibi 10 civarı küme.

Adımlar şöyledir:

1. k küme merkezi (centroid) rastgele ya da verilerden seçilerek atanır.
2. Her noktanın merkezlere uzaklığı ölçülür ve nokta en yakın merkezin kümesine atanır.
3. Her kümenin yeni merkezi, kümedeki noktaların ortalaması olarak hesaplanır.
4. 2. ve 3. adımlar tekrarlanır.

Durdurma için üç seçenek vardır: belirli bir adım sayısı, kümeler arasında yer değişikliği olmaması ya da küme içi varyansın en küçük, kümeler arası varyansın en büyük olması.

Kümenin adını algoritma vermez, insan verir. Sayısal verilerde bir doktor bakar: bütün hastaların insülin değerleri yüksekse “şeker hastalığı kümesi”, karaciğer enzimleri yüksekse “karaciğer hastalıkları” der. NLP’de içinde siyaset kelimeleri olan kümeye “siyaset”, spor kelimeleri olana “spor” etiketi verilir.

21.2 Diğer yöntemler

- **X-means:** k değerini algoritma kendisi belirler.
- **DBSCAN:** Doğrusal olarak ayrılamayan, iç içe geçmiş verilerde iyi sonuç verir.
- **Self-organizing map (SOM):** k-means’e benzer. k-means bir verinin bütün kümelere uzaklığını ölçerken SOM yalnızca belirli bir komşuluk mesafesindeki birimlere bakar. Tek ve iki boyutlu türleri vardır.

21.3 Yarı eğiticili yöntemler

Elde çok az etiket varsa etiketli yöntem kullanılamaz. Ama bu değerli etiketlerin kaybolması da istenmez. Bu durumda yarı eğiticili (semi-supervised) yöntemler kullanılır. En popüler olanı YATSI algoritmasıdır. Hoca açılımını hatırlayamadı.

> **Ders sonrası not:** YATSI’nın açılımı “Yet Another Two-Stage Idea”dır.

22. Metin Sınıflandırma

Hoca, metin sınıflandırma anlatılırken önceki detayların üzerinden yüzeysel geçileceğini söyledi. İçinde metin olan, sınıflandırılabilecek her şey bu konuya girer:

- Haber metinlerini başlığa ya da içeriğe göre ayırmak.
- Tweet’leri konuya ya da içerdiği duyguya göre ayırmak.
- Bir tweet’in nefret söylemi olup olmadığını bulmak.
- Metnin yazarını, yazarın cinsiyetini, dokümanın dilini bulmak.

- Bir kodun hangi programlama dilinde yazıldığını, içinde başka dillerde yazılmış kod olup olmadığını bulmak.
- İncelenen kodun exe kodu mu, normal kod mu olduğunu anlamak.
- Mailin spam olup olmadığını bulmak.

Makine öğrenmesi kullanılıyorsa eğiticili, eğiticisiz ve yarı eğiticili yöntemlerden biri seçilir.

> **Hocanın vurgusu:** Her özellik her uygulama için iyi sonuç vermez. Duygu analizi (sentiment analysis) yapılıyorsa duygu taşıyan kelimeler önemlidir, fiillerin belki hiç önemi yoktur. O zaman fiilleri özellik olarak almanın anlamı yoktur. Her zaman önce problemi keşfetmek gerekir.

Özellik adayları şunlardır: kelimeler, kelimelerin POS etiketleri, n-gram'lar, ekler ve türleri, noktalama işaretleri, yalnızca işlevsel kelimeler (function words). Doğrudan anlatım ifadeleri ("söyledi", "dedi", "ona göre") bile özellik olabilir.

23. Yazar Tanıma Özellikleri

Yazar belirleme çalışmasında kullanılabilecek özellikler şunlardır:

- **Cümle sayısı:** Bazı yazarlar uzun, bazıları kısa cümle kurar. Bir A4 sayfada 15 cümle de, 50 cümle de görülebilir. Bu, yazarın stilidir.
- **Toplam kelime sayısı (token):** Bazı yazarlar çok ek almayan kelimeler kullanır. Bazıları ise "uygarlaştıramadıklarımızdan mısınız" örneğindeki gibi çok çekim eki alan kelimeler kullanır. Bu yazarlarda kelime sayısı azdır.
- **Cümledeki ortalama kelime sayısı, kelime uzunluğu, farklı kelime sayısı.**
- **Kelime zenginliği (word richness):** Aynı kavram için farklı kelimeler kullanmak. Bir paragrafta "okul", diğerinde "mektep" denmesi, dilin zengin kullanıldığını gösterir.
- **Sözcük türleri:** Cümledeki isim, fiil, sıfat, zarf, edat, zamir, bağlaç sayıları.
- **Noktalama:** Nokta, virgül, iki nokta üst üste, soru işareti, ünlem sayıları.
- **Cümle yapısı:** Kurallı ve devrik cümle sayıları.
- **Üç nokta:** Bazı yazarlar cümleyi bitirmez, üç nokta koyup gerisini okura bırakır.

24. Metin Gösterimi ve Doküman-Terim Matrisi

Metin vektör hâline getirilir ve her özellik bir kelimeye karşılık gelir. Slaytta bir paragraftaki kelimelerin count değerleri gösteriliyor. Burada en sık kelime 3 kez geçtiyse, TF için 1 kez geçen kelimeye $1/3$ yazılır. Olasılık değerleri ya da ağırlıklandırılmış değerler de kullanılabilir.

Stop words, sayısal değerler, nokta, tire ve virgül atılabilir. Anlamları olmadığı için yalnızca vektörün boyunu uzatırlar.

Doküman-terim matrisinde satırlar dokümanlar, sütunlar terimler olabilir ya da tersi. d_{22} değeri, 2. dokümanda 2. terimin kaç kez geçtiğini söyler. Bütün terimler bütün dokümanlarda geçmeyeceği için bu seyrek bir veridir. Seyrek veriyle çalışmak zor olduğunda, gerekirse yumuşatma teknikleri uygulanır.

N-gram özellikleri kelime ya da karakter tabanlı olabilir. Eskiden token ayırma, kök bulma ve n-gram çıkarma için kod yazmak gerekirdi; hocanın doktorası sırasında bunların hiçbiri hazır değildi. Bugün Python kütüphaneleriyle tek satırda yapılıyor.

Slayttaki örnekte iki dokümandan kelime bigram'ları çıkarılmıştır. "army troops" 1. dokümanda bir kez geçmiş, 2. dokümanda hiç geçmemiştir. Özellik sayısını azaltmak için stop words silinir ve her şey küçük harfe çevrilir. Aksi hâlde "Army" ve "army" ayrı özellik olur.

25. Hiyerarşik Metin Kümeleme

Metinler de kümelenebilir. Etiketi bilinmeyen 9 doküman vektöre dönüştürülür. Başta her doküman ayrı bir küme kabul edilir. Benzerliklere bakılarak dokümanlar birleştirilir: önce D3 ile D7, sonra D1 ile D2 gibi. 4, 3 ya da 2 küme isteniyorsa ilgili seviyede kesilir.

Bu hiyerarşinin bir faydası vardır. Bazı dokümanlar birden fazla konu içerebilir. D3 ve D7 siyasetle, D9 ekonomiyle ilgili olsun, D4 ise ikisini birden içersin. Siyaset ve ekonomi iç içe

olduğu için düz bir ayırma başarılı olmayabilir. Hiyerarşik yapıda farklı seviyeler incelenebilir, ayrıntı görülmek istenirse alt seviyeye inilir.

Durdurmak için bir eşik verilebilir: kümeler arası benzerlik eşiğin altına düşünce birleştirme durur. Kümeler birleştirilirken benzerlik için single link, complete link ve group average link gibi yaklaşımlar vardır. Ayrıntıları veri madenciliği dersinde görülür.

Kümeleme yukarıdan aşağı (top-down) ya da aşağıdan yukarı (bottom-up) yapılabilir. Birincisinde her şey tek kümedeyken ayrılır, ikincisinde her doküman ayrı kümedeyken birleştirilir.

26. Boyutun Laneti ve Özellik Kararları

Çok boyutlu vektörlerle işlem süresi çok artar. Hocanın çok önemli dediği yeşil kutu, boyut sayısına göre verilerin merkez noktasına yakınlığını gösteriyor. Veri tek boyutluysa verinin %50'si bir noktada toplanır ve sınıflandırma ya da kümeleme yapmaya kalkınca ayrım yapılamaz. Boyut sayısı arttıkça ise veriler üstel olarak birbirinden uzaklaşır ve yine zorlanılır.

Boyut ile örnek sayısı arasında teorik bir ilişki vardır: 1 boyut için 10, 2 boyut için 100, 3 boyut için 1000, 4 boyut için 10.000 örnek. Üstel olarak artar. Bu teoridedir, pratikte buna ulaşılamaz. Ama her zaman bir orana dikkat edilmelidir:

- Örnek sayısı azsa boyut sayısı azaltılmalıdır.
- Boyut sayısı azken örnek çok fazlaysa, örnekler rastgele seçilerek azaltılabilir.

Özellikler belirlenirken verilmesi gereken kararlar şunlardır:

- Kök mü, gövde mi, kelimenin tamamı mı kullanılacak?
- Stop words ve function words atılacak mı?
- Filtre ya da sarmalayıcı yöntemle özellik seçilecek mi?
- Uzayı değiştiren yöntemler kullanılacak mı? LDA sınıf bilgisi kullanır. LSI ve PCA'de sınıf bilgisi yoktur.

Çok seyrek kelimeler de atılabilir. Doküman türü belirlenirken derlemde bir kez geçen kelime pek yardımcı olmaz. Yazar tanımada ise tek bir yazarın kullandığı düşük frekanslı kelimeler çok ayırt edici olabilir.

Hoca, kelimeler frekanslarına göre dağıtıldığında elde edilen bir grafiği gösterdi. Yüksek frekanslı önemsiz kelimeler stop words'tür. Düşük frekanslı önemsiz kelimeler kişiye özel kullanımlar olabilir. Ortadaki kelimeler ise önemlidir. Hangi bölgenin önemli olduğu yine uygulamaya bağlıdır.

> **Ders sonrası not:** Hoca önce "sol taraf önemsiz ve düşük frekanslı, stop words" dedi, ardından stop words'ü doğru biçimde yüksek frekanslı uca yerleştirdi. Hoca grafiği "güzel bir normal dağılım" diye de nitelendirdi; kelime frekansları normal dağılıma değil, Zipf yasasına benzer bir dağılıma uyar.

27. Kelime Torbası

Derlemdeki bütün tekil kelimelerden oluşan yapıya, yani sözlüğe, kelime torbası (bag of words) denir. Dokümanda geçen bütün kelimelerin içinde bulunduğu bir çanta gibi düşünülebilir.

28. Abstract Feature Extraction

28.1 Yöntemin hikâyesi

Hoca bu yöntemin kitaplarda bulunmadığını söyleyip hikâyesini anlattı. Bir lisans dersinde verilen ödevde bir öğrenci henüz çok gelişmiş olmayan bir şey yapmış ve güzel bir sonuç almış. Ulusal bir yayın yapılmış, sonra unutulmuş.

Yıllar sonra Göksel Biricik doktorasında takılınca, hoca bu çalışmayı hatırlamış ve onun üzerinde deneme yapmasını önermiş. Yöntemin ilk adı "Yıldız"mış. Sempozyumlara gönderilmiş, ama dergiler sonuçların bu kadar iyi olmasına inanmamış: "Kendi veri setinizde iyi çalışıyor" demişler. Hakemlerin isteğiyle sırasıyla Reuters (o dönem postayla CD olarak gelmiş), ModApte ve 20 Newsgroups veri setlerinde denenmiş. Her seferinde iyi sonuç alınmış ve sonunda

çalışma kabul edilmiş. O dönemde derin öğrenme yoktu, makine öğrenmesi yeni yeni gündeme geliyordu.

28.2 Temel fikir

Bilgi erişimi konusunda anlatılacak öz vektör, öz değer ve tekil değer ayrıştırması (Singular Value Decomposition) gibi güçlü bir matematik bu yöntemde kullanılmıyor. Yalnızca terimlerin ağırlıkları ve bu terimlerin sınıflar üzerindeki olasılık dağılımları kullanılıyor.

Gösterim şöyledir: i terim, j doküman, k sınıf indisidir.

- $n_{i,j}$: t_i teriminin d_j dokümanında kaç kez geçtiği.
- N_i: t_i terimini içeren doküman sayısı.
- N: derlemdeki toplam doküman sayısı.

Formüller şöyledir:

Terimin bir sınıftaki dokümanlarda kaç kez geçtiği:

$$nc_{i,k} = \sum_{j} n_{i,j}, \quad d_j \in c_k$$

Terimin sınıfı ne kadar etkilediği. Terim bir sınıfta hiç geçmeyebileceği için "+1" eklenir (add-one yumuşatması). $\log(N/N_i)$ normalizasyon katsayısıdır. Hoca, başka katsayıların da kullanılabileceğini, kendi verilerinde bunun uygun çıktığını söyledi:

$$w_{i,k} = \log(nc_{i,k} + 1) \times \log\left(\frac{N}{N_i}\right)$$

Bir dokümandaki terimlerin c_k sınıfına toplam etkisi:

$$Y_{j,k} = \sum_{t_i \in d_j} w_{i,k}$$

Normalize edilmiş yeni özellik:

$$yeniTerim_{j,k} = \frac{Y_{j,k}}{\sum_k Y_{j,k}}$$

28.3 AFE örneği

Örnek 1. İki sınıfta toplam 6 doküman ve 8 terim var.

- Sınıf 1: B1 (t1, t2, t3), B2 (t2, t4, t8), B3 (t2, t2, t5).
- Sınıf 2: B4 (t5, t6), B5 (t5, t7, t8), B6 (t2, t5, t6).

Adım 1: Terim-doküman matrisi.

Terim	d1	d2	d3	d4	d5	d6
Terim1	1	0	0	0	0	0
Terim2	1	1	2	0	0	1
Terim3	1	0	0	0	0	0
Terim4	0	1	0	0	0	0
Terim5	0	0	1	1	1	1
Terim6	0	0	0	1	0	1
Terim7	0	0	0	0	1	0
Terim8	0	1	0	0	1	0

Matris seyrektir ama bu, yöntem için sorun değildir.

Adım 2: Terim ağırlıkları (logaritmalar 10 tabanında). Terim 5 için $N = 6$, $N_5 = 4$. Terim sınıf 1'de 1 kez, sınıf 2'de 3 kez geçiyor.

$$w_{5,1} = \log(1+1) \times \log(6/4) \approx 0{,}053$$

$$w_{5,2} = \log(3+1) \times \log(6/4) \approx 0{,}106$$

Terim 6 için $N_6 = 2$. Terim sınıf 1'de hiç, sınıf 2'de 2 kez geçiyor.

$$w_{6,1} = \log(1) \times \log(3) = 0$$

$$w_{6,2} = \log(3) \times \log(3) \approx 0{,}228$$

Aynı işlem 8 terimin hepsi için yapılır. Artık her terimin her sınıfı ne kadar temsil ettiği bilinir. Ama amaç terimleri değil dokümanları sınıflandırmaktır.

Adım 3: Doküman 4. Bu dokümanda t5 ve t6 vardır.

$$Y_{4,1} = 0{,}053 + 0 = 0{,}053$$

$$Y_{4,2} = 0{,}106 + 0{,}228 = 0{,}334$$

$$yeniTerim_{4,1} = \frac{0{,}053}{0{,}053 + 0{,}334} \approx 0{,}137$$

$$yeniTerim_{4,2} = \frac{0{,}334}{0{,}387} \approx 0{,}863$$

Yorumu şudur: doküman 4, yaklaşık %14 sınıf 1'e, %86 sınıf 2'ye aittir.

Adım 4: Bütün dokümanlar.

Doküman	Özellik 1 (Sınıf 1)	Özellik 2 (Sınıf 2)
Doküman 1	0,918	0,082
Doküman 2	0,718	0,282
Doküman 3	0,585	0,415
Doküman 4	0,137	0,863
Doküman 5	0,289	0,711
Doküman 6	0,313	0,687

Doküman 1-3 sınıf 1'de, doküman 4-6 sınıf 2'de daha yüksek değer alıyor. Yani doğru ayrılıyorlar. Grafikte de bu görülür.

> **Ders sonrası not:** Tablodaki değerler python ile yeniden hesaplandı ve tuttu. Doküman 3'ün 0,585 değeri, t2 terimi iki kez geçtiği için ağırlığı iki kez toplanırsa çıkar ($0{,}299/0{,}511$). Terim tek sayılırsa 0,525 çıkar. Yani $Y_{j,k}$ toplamında terimin dokümandaki her geçişi ayrı sayılmıştır.

28.4 Değerlendirme

AFE aynı zamanda bir özellik çıkarımıdır. Özellik sayısını sınıf sayısına indirir: burada 8 terimden 2 özelliğe.

Avantajları şunlardır: etiketli veri varsa kullanılabilir, arkasında ağır bir matematik yoktur, boyutu kesinlikle sınıf sayısına indirir.

Dezavantajı şudur: gelen dokümanın hiçbir terimi eğitim verisinde yoksa her sınıfa %0 ağırlık çıkar. Bu, bambaşka konuda bir dokümandır ve sınıflandırılamaz. Bu durumda doküman derleme katılır, sistem yeniden eğitilir ve içindeki terimler de ağırlık kazanır. Dile yeni terimler de girdiği için sistemi aralıklarla yeniden eğitmek gerekir.

29. Kelime Koordinatları Yöntemi

Hoca, anlamsal ilişkileri de özellik olarak kullanabileceğimizi hatırlattı. İki kelimenin ilişki türü bilinmese bile (zıtlık, eş anlamlılık, alt kavram, üst kavram) ne kadar benzediği bulunabilir. Embedding varsa vektörlerin benzerliğine bakılır. Embedding yoksa "kelime koordinatları" yöntemi kullanılabilir.

İki kelime arama motoruna verilir. Arka arkaya kaç kez geçtiklerine, aynı cümlede ya da aynı dokümanda bulunup bulunmadıklarına bakılır. Hangisinin kullanılacağı tercihe bağlıdır. Örneğin akbaba ve baykuş Google'a verildiğinde 20 eşleşme dönmüş.

	akbaba	ayı	baykuş	araba	limuzin
akbaba		0	20	1	0
ayı	0		33	4	0
baykuş	20	33		0	0
araba	1	4	0		38
limuzin	0	0	0	38	

Bu matris çok boyutlu ölçekleme (Multidimensional Scaling, MDS) ile iki boyutlu düzleme yerleştirilir. MATLAB'de kütüphanesi vardır. Sonuçta limuzin ve araba bir tarafta (taşıt), ayı, baykuş ve akbaba diğer tarafta (hayvan) toplanır. Yalnızca birlikte geçme sayılarıyla bile ayrım yapılabilir.

Deneyler genişletilmiş:

- **Veri kümesi 1:** Yer, hayvan, taşıt (Google). Sınıflar ayrılıyor. Örneğin "at" kelimesi, birlikte kullanıldığı için taşıt kümesine de yakın çıkıyor.
- **Veri kümesi 2:** Ev eşyası, hayvan, giyecek. "Merdiven" ve "divan" ortalarda bir yere kaymış.
- **Veri kümesi 3:** 6 sınıf (yiyecek, giyecek, ev eşyası, hayvan, yer, taşıt), hem Yahoo hem Google ile. Google ile sonuçlar daha iyi, çünkü indeksleme mekanizması daha iyidir.

İyileştirme. Akbaba ile ayı ikisi de hayvan olduğu hâlde birlikte hiç geçmedikleri için 0 değeri alıyor. Oysa aynı sınıfta olmaları gerekir. Bu yüzden ham sayım yerine normalize edilmiş bir değer kullanılır:

$$\frac{n(A,B)}{n(A) \times n(B)}$$

Hoca bir örnek verdi: A 3 kez, B 5 kez, ikisi birlikte 7 kez geçmişse değer $\frac{7}{3\times5} \approx 0{,}47$ olur.

> **Ders sonrası not:** Bu sayılar yalnız formülü göstermek içindir. Birlikte geçme sayısı, kelimelerin tek başına geçme sayısından büyük olamaz.

Farklı sınıflandırma yöntemleri ve boyutlarla sonuçlar çok değişiyor. Aynı veri setinde iki boyutta Expectation Maximization çok daha iyi sonuç verirken, çok boyutta aynı başarıyı göstermemiş.

Hocanın vurgusu: "Pat diye" bir yöntem seçip kullanmamak gerekir. Çapraz geçerleme ve yöntem denemek önemlidir.

30. TF-IDF

Şimdiye kadar hep TF'den söz edildi. IDF, Inverse Document Frequency yani ters doküman frekansıdır.

Örnek 2. Dört doküman olsun.

Kelime	D1	D2	D3	D4	Toplam	Geçtiği doküman
A	7	3	5	6	21	4
B	1	0	0	10	11	2

Yalnızca terim frekansına bakılırsa A daha önemli görünür. Ama A her dokümanda geçtiği için ayırt edici değildir. TF-IDF şöyle hesaplanır (N toplam doküman sayısı, n kelimenin geçtiği doküman sayısı; hocanın hesabında logaritma 2 tabanındadır):

$$w = tf \times \log \frac{N}{n}$$

$$w_A = 21 \times \log_2(4/4) = 21 \times 0 = 0$$

$$w_B = 11 \times \log_2(4/2) = 11 \times 1 = 11$$

Demek ki B, A'dan daha ayırt edicidir. B'nin geçtiği dokümanlar, geçmediği dokümanlardan çok farklıdır.

Hocanın vurgusu: Bazı uygulamalarda ham terim frekansı yerine TF-IDF kullanmak çok önemlidir. Hangisinin daha iyi olduğu denenerek bulunur.

31. İzdüşüm Yöntemleri

AFE, PCA, LDA ve bilgi erişiminde anlatılacak LSI (Latent Semantic Indexing) boyutu değiştiren, yani izdüşüm (projeksiyon) alan yöntemlerdir.

Slayttaki örnekte kırmızı ve mavi noktalar var. Bir eksene izdüşürülünce birbirine karışıyorlar. Uzay değiştirilip başka bir eksene izdüşürülünce kırmızılar bir tarafta, maviler diğer tarafta toplanıyor ve kolayca ayrılıyor. Bu yüzden uzayı değiştirmek bazen önemlidir.

Kelimeler frekanslarına göre renklendirilerek de görselleştirilebilir. Mavi kelimeler az, koyu renkli uçtakiler çok sık geçen kelimelerdir.

32. Naive Bayes ile Metin Sınıflandırma

Hoca, seçilen projelere bakınca birçok öğrencinin bu yöntemi tercih edeceğini gördüğünü söyledi. Amaç, gelen dokümanın her sınıfı hangi olasılıkla temsil ettiğini bulmak ve en yüksek değeri veren sınıfı seçmektir:

$$v_{NB} = \arg\max_{v_j \in V} P(v_j) \prod_i P(w_i|v_j)$$

$P(v_j)$ sınıfın olasılığıdır. Çarpım, terimlerin o sınıftaki olasılıklarının çarpımıdır. Terim frekansı sıfır olabileceği için add-one yumuşatması uygulanır:

$$P(w_k|v_j) = \frac{n_k + 1}{n_j + |V|}$$

n_k, w_k kelimesinin j. sınıftaki sayısıdır. n_j, j. sınıftaki kelime sayısıdır. $|V|$, sözlükteki kelime sayısıdır.

Örnek 3. İki eğitim dokümanı var.

- Sınıf A: “cat”, “crab”, “crawl” ve “stairs” kelimelerini içeren bir cümle.
- Sınıf B: “It’s raining cats and dogs”.
- Test dokümanı: içinde “cats” ve “dogs” geçen bir cümle.

Adım 1. Stop words ve ekler atılır.

- Sınıf A kelimeleri: cat, crab, crawl, stair (4 kelime).
- Sınıf B kelimeleri: rain, cat, dog (3 kelime).

Adım 2. Ortak sözlük oluşturulur: cat, dog, crab, crawl, stair, rain. $|V| = 6$.

Adım 3. Sınıf olasılıkları hesaplanır. Toplam 2 doküman var ve her sınıfta 1 tane bulunuyor: $P(A) = 1/2$, $P(B) = 1/2$.

Adım 4. Test dokümanında sözlükte bulunan kelimeler yalnız cat ve dog’dur. Hesap bu iki kelimeyle yapılır.

Adım 5. A sınıfı için:

$$P(\text{cat}|A) = \frac{1+1}{4+6} = \frac{2}{10}, \quad P(\text{dog}|A) = \frac{0+1}{4+6} = \frac{1}{10}$$

$$P(A) \times P(\text{cat}|A) \times P(\text{dog}|A) = \frac{1}{2} \times \frac{2}{10} \times \frac{1}{10} = 0{,}01$$

Adım 6. B sınıfı için:

$$P(\text{cat}|B) = P(\text{dog}|B) = \frac{1+1}{3+6} = \frac{2}{9}$$

$$P(B) \times P(\text{cat}|B) \times P(\text{dog}|B) = \frac{1}{2} \times \frac{2}{9} \times \frac{2}{9} \approx 0{,}0247$$

Sonuç. $0{,}0247 > 0{,}01$ olduğu için test dokümanı B sınıfına aittir.

Bu, Naive Bayes’in anlatılabilecek en kısa hâlidir. Multinomial Naive Bayes özellik değerlerini algoritma içinde değiştirir ve bazen daha iyi sonuç verir.

Dersin Toparlaması

Bu derste bir kelimenin nasıl sayıya dönüştürüleceğiyle başladık. One-hot gösteriminde kelimeler arasında benzerlik yoktur. Doküman, bağlam ve trigram tabanlı gösterimler benzerliği yakalamaya başlar, ama vektörler uzun ve seyrektir. word2vec, FastText, GloVe ve BERT ise düşük boyutlu ve yoğun vektörler üretir. Benzerlik için metinlerde en çok cosine kullanılır.

Metin sınıflandırma tarafında ise izlenecek yol şöyle özetlenebilir: doğru özellikler seçilir, özellik ile örnek sayısı arasındaki oran ayarlanır, gereksiz özellikler özellik seçimiyle atılır. Veriler çok örtüşüyorsa izdüşüm yöntemleriyle başka bir uzaya taşınır. Her yöntem ailesinden birer yöntem denenir, eğitim ve testte çapraz geçerleme yapılır. Hocanın sonucu şudur: bunlar düzgün yapılırsa kötü sonuç almak zordur, ama veri setinin içeriği de çok önemlidir. LSI ve öz vektörler bilgi erişimi konusunda anlatılacaktır.

Ders Sonrası Notlar

> **Ders sonrası not:** Hoca uzaklık için “bir bölü benzerlik” dedi. $[0, 1]$ aralığında genelde uzaklık $d = 1 - s$ alınır. $1/s$ kullanılırsa benzerlik 0 iken uzaklık sonsuz olur; hocanın söylediği “benzerlik 0 ise uzaklık 1” ifadesine uyan biçim $1 - s$’tir.

Ders sonrası not: Hoca PMI tablosunda kelimenin kendisiyle benzerliğini 1 olarak okudu ve 0,16 gibi değerleri "benzerlik katsayısı" diye açıkladı. Formüldeki PMI değeri 1 ile sınırlı değildir. Slayttaki katsayılar büyük olasılıkla PMI değerlerinden oluşturulmuş vektörlerin benzerliğidir; slayt olmadan bu doğrulanamadı. Hoca sydney-melbourne için 0,43 dedi, kısa notta slayttan alınan değer 0,44'tür.

Ders sonrası not: word2vec, GloVe ve LDA'nın hepsinde "arka planda matris ayrıştırma yapılır" ifadesi tam doğru değildir. word2vec bir sinir ağıyla eğitilir ve matrisi açıkça ayrıştırmaz; GloVe birlikte geçme matrisi üzerinde çalışır. Hocanın "GloVe, word2vec'in optimize edilmiş hâlidir" ifadesi de ancak yaklaşık doğrudur, çünkü GloVe ayrı bir modeldir.

Ders sonrası not: Hoca BERT derlemi için "8 milyon kelimelik" dedi. BERT, yaklaşık 800 milyon kelimelik BooksCorpus ile 2,5 milyar kelimelik İngilizce Wikipedia üzerinde eğitilmiştir.

Ders sonrası not: Hoca bir yerde "LSI, PCA sınıf bilgisi olan yöntemler" dedi, hemen ardından düzeltti. Doğrusu: PCA ve LSI sınıf bilgisi kullanmaz, LDA kullanır.

Ders sonrası not: Naive Bayes sonucunda hoca bir yerde "0,247 daha büyük" dedi. Doğru değer 0,0247'dir. Hesap python ile doğrulandı: $0{,}01$ ve $0{,}0247$.

Ders sonrası not: Altyazıda bozuk geçen terimlerin doğruları: "SIBO/Sibov" = CBOW, "hatvan/van hat" = one-hot, "Mikalov" = Mikolov, "Glow/Globe" = GloVe, "Ludus" = Loodos, "Rapper/röper" = wrapper, "Vega" = Weka, "K-MINS/Kamins" = k-means, "Rolter/Reuter" = Reuters, "ModUp/mod apte" = ModApte, "Nayibayes" = Naive Bayes, "Advanced smoothing" = add-one smoothing, "AFI" = AFE, "Alberto, Roberto" = ALBERT, RoBERTa.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.